



Repaso teórico primer parcial

Unidad 1: Ingeniería de software en contexto

- ☒ ~~Introducción a la Ingeniería del Software. ¿Qué es?~~
- ☒ ~~Estado Actual y Antecedentes. La Crisis del Software.~~
- ☒ ~~Disciplinas que conforman la Ingeniería de Software.~~
- ☒ ~~Ejemplos de grandes proyectos de software fallidos y exitosos.~~
- ☒ ~~Ciclos de vida (Modelos de Proceso) y su influencia en la Administración de Proyectos de Software.~~
- ☒ ~~Procesos de Desarrollo Empíricos vs. Definidos.~~
- ☒ ~~Ciclos de vida (Modelos de Proceso) y Procesos de Desarrollo de Software~~
- ☒ ~~Ventajas y desventajas de c/u de los ciclos de vida.~~
- ☒ ~~Criterios para elección de ciclos de vida en función de las necesidades del proyecto y las características del producto.~~
- ☒ ~~Componentes de un Proyecto de Sistemas de Información.~~
- ☒ ~~Vínculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software~~
- ☒ ~~No silver bullet~~

Unidad 2: Gestión Lean-Agile de Productos de Software

- ☒ ~~Gestión de Producto~~
- ☒ ~~Requerimientos Ágiles~~
- ☒ ~~User Stories~~
- ☒ ~~Estimaciones Ágiles~~
- ☐ Framework Scrum

UNIDAD 1: Ingeniería de software en contexto

Introducción a la Ingeniería del Software. ¿Qué es?

El **software** no es solo código, sino el conjunto de programas junto con toda la documentación requerida (manuales, arquitectura) y las herramientas utilizadas para construirlo.

La **Ingeniería de Software** es la disciplina que se encarga de todos los aspectos de la producción de **software**, desde la especificación hasta el mantenimiento del mismo, buscando desarrollar sistemas confiables y económicos de manera rápida.

Estado Actual y Antecedentes. La Crisis del Software.

Sirvió para describir la dificultad que existía en generar software sin defectos, comprensible y verificable.

Causas principales de la crisis:

- **Aumento de la potencia del Hardware:** Las máquinas se volvieron órdenes de magnitud más potentes, permitiendo problemas más complejos que los programadores no sabían cómo estructurar.
- **Enfoque Artesanal:** El software se escribía sin metodologías. No había fases de análisis o diseño; se pasaba directamente al código.
- **Falta de Disciplina:** No existían estándares de calidad ni procesos de estimación. Los proyectos se entregaban tarde, con errores graves y muy por encima del presupuesto.
- **Mantenimiento Imposible:** El código era tan enredado que modificarlo era más caro que hacerlo de nuevo.

Esto ayudó a entender que en la actualidad, los proyectos suelen fracasar por dos factores principales:

- las demandas crecientes (necesidad de desarrollos cada vez más rápidos y complejos).
- las bajas expectativas (empresas que se conforman con que el software "funcione" sin aportar verdadero valor al negocio).

Disciplinas que conforman la Ingeniería de Software

Se dividen en tres grandes grupos:

- **Técnicas:** aportan directamente al desarrollo del producto (toma de requerimientos, análisis, diseño, implementación, prueba y despliegue).
- **De soporte:** son disciplinas transversales a los procesos de software, que permiten **verificar la integridad y calidad de un producto** de software
- **De Gestión:** hacen referencia a **actividades de planificación, monitoreo y control del proyecto** de desarrollo de software

Ejemplos de grandes proyectos de software fallidos y exitosos.

Un **software exitoso** se da cuando hay involucramiento del usuario, apoyo de la gerencia, requerimientos correctamente enunciados y expectativas realistas.

Un **proyecto fallido** ocurre cuando hay requerimientos incompletos o cambiantes, falta de recursos, mala planificación, o cuando la versión final no satisface las necesidades del cliente excediendo tiempos y costos.

Ciclos de vida (Modelos de Proceso) y su influencia en la Administración de Proyectos de Software.

Se denomina **ciclo de vida** a una serie de pasos a través de los cuales un **producto** o un **proyecto** progresa en el tiempo. Es decir, que es una **representación simplificada de un proceso**, el cuál define **elementos** (actividades estructurales, productos del trabajo, tareas, etc.) y el **flujo del proceso**, que especifica la relación y el orden de dichos elementos.

Son modelos genéricos de los procesos de software, y establecen los criterios que utiliza para determinar si se debe pasar de una tarea a la siguiente.

En otras palabras, el ciclo de vida especifica:

- Las fases del proceso (requerimientos, especificaciones, diseño).
- Orden en el cual se llevan a cabo.

Relación entre el ciclo de vida del proyecto y del producto:

El **ciclo de vida del producto siempre es mayor que el ciclo de vida del proyecto**, ya que el ciclo de vida del proyecto dura lo que dura el desarrollo del software. En cambio, el ciclo de vida del producto dura hasta que el software se deje de utilizar, dependiendo esto de la madurez que tenga el producto en base a las necesidades del mercado.

Clasificación de los ciclos de vida:

- **Modelo Secuencial:** es la forma lineal. El proyecto progresa a través de una secuencia ordenada de pasos (fases) y una actividad no puede iniciar sin que la precedente haya sido finalizada.
 - Ventajas: minimiza gastos de planificación al hacerla toda por adelantado, genera documentación útil y encuentra errores tempranos por fase.
 - Dificultades: exige que los requerimientos sean completamente explícitos desde un principio. Burocrático, no lidia con la incertidumbre, el cliente obtiene una versión funcional del producto muy tarde y encontrar un defecto implica un gran rediseño en la solución.
- **Modelo iterativo/incremental:** este modelo aplica sucesivas iteraciones en forma escalonadas a medida que se avanza en el proyecto. Cada iteración produce como resultado una parte del software totalmente funcional y potencialmente entregable.
 - Ventajas: rápida retroalimentación del cliente, permite adaptación a requerimientos cambiantes y el cliente obtiene valor temprano.
 - Desventajas: el proceso se invisibiliza un poco, documentar cada iteración es costoso y la arquitectura puede degradarse con tantos incrementos
- **Modelo recursivo:** Orientado a gestionar riesgos en sistemas muy complejos, resolviendo los riesgos más grandes en mini-proyectos iterativos.

- Ventajas: excelente para manejar riesgos masivos
- Desventajas: muy costoso, puede carecer de documentación estructurada y la versión final se ve recién al final del ciclo.

Consta de las siguientes etapas:

1. Determinar objetivos.
2. Identificar y resolver los riesgos.
3. Desarrollar y probar.
4. Planificar la próxima iteración.

Procesos de Desarrollo Empíricos vs. Definidos.

Proceso de desarrollo de software: conjunto de **actividades, métodos, prácticas y transformaciones** que la gente usa para **desarrollar o mantener software**.

Factores determinantes:

1. **Procedimientos y Métodos:** deber estar definidos para asegurar la calidad del resultado.
2. **Personas motivadas, capacitadas y con habilidades:** el esfuerzo de las personas es crucial.
3. **Herramientas y Equipos:** materiales necesarios para llevar a cabo el proceso

Actividades fundamentales:

1. **Especificación de software**
2. **Desarrollo**
3. **Validación**
4. **Evolución**

Procesos Definidos

Inspirados en líneas de producción industrial. Son **deterministas** (asumen que a misma entrada, misma salida). Su administración y control se basa en la predictibilidad, planes estrictos y mucha documentación.

Ante la misma entrada se pretende que se obtendrá la misma salida. Asumen que, si se aplican una y otra vez, **se obtendrán siempre los mismos**

resultados.

Procesos Empíricos

Basados en la **experiencia**. No asumen que se obtendrán los mismos resultados repitiendo el proceso, ya que dependen del contexto y la creatividad. Utilizan ciclos cortos guiados por tres pilares:

- **Transparencia**
- **Inspección**
- **Adaptación**

Se basan en ciclos cortos de inspección y adaptación (**retroalimentación**), porque ante un análisis, se ajustan de mejor forma. La administración y el control es por medio de **inspecciones frecuentes** y **adaptaciones** para lograr buenas prácticas.

Criterios para elección de ciclos de vida en función de las necesidades del proyecto y las características del producto.

Se debe evaluar la incertidumbre, claridad de los requisitos y urgencia del cliente:

Elegir **Secuencial** si los requerimientos son muy claros y estables, hay baja incertidumbre y el cliente no necesita entregas rápidas parciales.

Elegir **Iterativo/Incremental** si hay volatilidad de requerimientos (alta incertidumbre) y el cliente necesita ver resultados/valor funcional de forma temprana.

Elegir **Recursoivo** si el proyecto es a gran escala y el objetivo principal es disminuir riesgos técnicos.

Ciclo de Vida	Procesos de desarrollo que lo admite	Características que lo hacen elegible
Secuencial	Definidos	• Alta certeza (baja incertidumbre). • La organización tiene una forma tradicional de trabajar. • No se pueden realizar entregas parciales del software (se debe entregar todo junto). • Eliminar riesgos de planificación.
Iterativo/incremental	Definidos o Empíricos	• Existe incertidumbre. • Volatilidad de requerimientos. • Posibilidad de entregas parciales. • Uso en etapas tempranas del producto.
Recursoivo	Definidos	• Mucho control de riesgos en cada iteración. • No se pueden realizar entregas parciales.

Componentes de un Proyecto de Sistemas de Información.

Un proyecto es un esfuerzo **temporal** (tienen una duración limitada), **único** y **orientado a objetivos** medibles para crear un producto. Sus componentes principales son:

Triple restricción: balance entre Costo, Tiempo y Alcance (trabajo que debe llevarse a cabo).

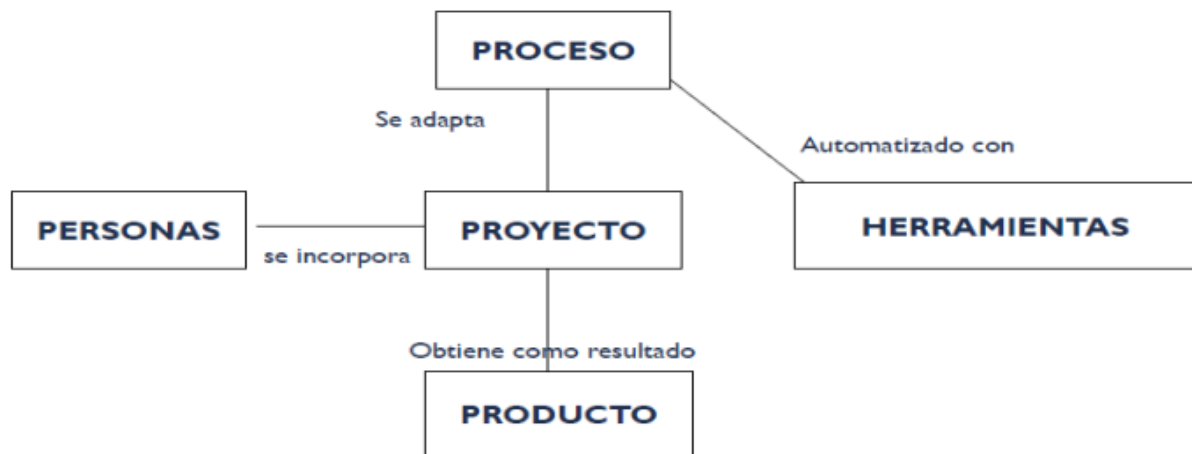
Roles:

- **Líder de Proyecto:** coordina y administra. Algunas de sus responsabilidades son:
 1. Definir el alcance del proyecto.
 2. Identificar a los involucrados, recursos y presupuesto.
 3. Detallar las tareas, estimar los tiempos y requerimientos.
 4. Identificar y evaluar riesgos.
 5. Preparar planes de contingencia.
 6. Controlar hitos y participar en las revisiones de las fases del proyecto.
 7. Administrar el proceso de control de cambios.
 8. Producir reportes de estado.
- **Equipo de Proyecto:** grupo de personas comprometidas con alcanzar un conjunto de objetivos de los cuales se sienten mutuamente responsables.

- Stakeholders: involucrados/interesados.

Plan de Proyecto: Hoja de ruta que define el alcance, estimaciones (tamaño, esfuerzo, costo, tiempo), gestión de riesgos y métricas. En el se documentan todas las decisiones que se toman a lo largo del desarrollo; en términos de: el proceso, la división de tareas a realizar, asignación de responsabilidades, equipos de trabajo, calendario de desarrollo, actividades de soporte (planes de contingencia, mecanismos para control de cambios, evaluar calidad y valorar riesgos).

Vínculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software.



El **proceso**, automatizado con **herramientas**, se adapta al **proyecto**, al cual se incorporan **personas**, y de este se obtiene como resultado un **producto**.

El Proceso (el marco conceptual):

- Funciona como un marco conceptual o una "plantilla" que establece el plan completo para desarrollar el software.
- Define qué **actividades**, **métodos** y **prácticas** deberían hacerse para transformar los requerimientos en un producto final.
- Dado que **no existe un proceso ideal que sirva para cualquier caso**, este marco nunca se aplica de forma estricta, sino que proporciona las bases para que las disciplinas de gestión lo adapten a las características específicas en el proyecto y en lo que se va a construir.

El Proyecto (la instancia organizativa):

- Es la **unidad organizativa o de gestión**. Es un esfuerzo temporal que requiere el acuerdo y la coordinación de un conjunto de especialidades y recursos para la creación de un producto, servicio o resultado único.
- Su función principal es "instanciar" o materializar el proceso. Toma la teoría del proceso y la **adapta** a la realidad y necesidades concretas, administrando los recursos y las personas con las que se cuenta.
- En enfoques tradicionales, antes de que el proyecto pueda definirse, es necesario determinar los **objetivos** y el **alcance** del producto, ya que esto permitirá realizar las estimaciones pertinentes para el proyecto.

Las Personas (el factor fundamental):

- Son la pieza más importante, debido a que el desarrollo de software es una actividad humano-intensiva.
- Aunque se adopte el "mejor proceso" teórico y se destine muchísimo esfuerzo a planificar el proyecto, **de nada servirá si no se cuenta con un equipo adecuado**, con las habilidades y capacidades necesarias para llevarlo a cabo.
- El proyecto incorpora a estas personas asignándoles **roles bien definidos** para que asuman la responsabilidad de ejecutar las actividades del proceso.

Las Herramientas (los facilitadores):

- Se utilizan para **automatizar, asistir y facilitar** las actividades que el proceso exige o necesita (por ejemplo, utilizar un software de diseño para realizar diagramas de clases o bases de datos).

El Producto (el resultado final):

- Es el bien o servicio final que se obtiene como fruto de todo este esfuerzo conjunto.
- Las características y la complejidad del producto son las que dictaminan **qué tipo de proceso** deberá adoptarse y cómo deberá gestionarse el proyecto.

En resumen: El **proceso** te dice cómo deberías trabajar en teoría; el **proyecto** toma esa teoría y la organiza para tu caso real; le sumas **herramientas** y **personas** capacitadas trabajando bajo roles claros, y como resultado de esa sinergia obtienes el **producto** de software

UNIDAD 2: Gestión Lean-Agile de Productos de Software

Valores ágiles

1. **Individuos en interacciones, por sobre procesos y herramientas:** Es preferible un equipo motivado y comunicado con herramientas pobres, que un equipo desmotivado con las mejores herramientas. Los procesos deben adaptarse al equipo y no al revés
2. **Software funcionando por sobre documentación extensiva:** Priorizar el software funcionando asegura entregar valor real al cliente para obtener retroalimentación constante. Solo se debe documentar lo estrictamente necesario.
3. **Colaboración con el cliente por sobre negociación contractual:** Fomenta que el equipo y el cliente trabajen juntos para descubrir las verdaderas necesidades (que suelen cambiar), en lugar de apegarse rígidamente a un contrato firmado al inicio con requerimientos que podrían volverse obsoletos.
4. **Responder a cambios por sobre seguir un plan:** Es imposible conocer al 100% las necesidades al inicio. El equipo debe ser flexible para adaptarse rápidamente a los cambios en el proyecto en lugar de ejecutar a ciegas un plan predefinido. **Los principales valores de la gestión ágil son la inspección y la adaptación**, diferentes a los de la gestión de proyectos tradicional: planificación y control que evite desviaciones del plan.

Principios ágiles

1. **Nuestra mayor prioridad es satisfacer al cliente:** La mayor prioridad es satisfacer al cliente a través de entregas tempranas y continuas de software con valor para él.
2. **Aceptar que los requisitos cambien, incluso en etapas tardías de desarrollo:** Se aceptan y aprovechan los cambios en los requerimientos,

incluso en etapas tardías del desarrollo, y esto permite que el equipo sea flexible, y responda y se adapte siempre ante las necesidades cambiantes.

3. **Entregar software funcional frecuentemente:** Es imprescindible que un equipo ágil **al finalizar cada iteración**, haya sido capaz de **transformar** uno o más requerimientos en **código** desarrollado, testeado y potencialmente desplegable.
4. **Técnicos y no técnicos trabajando juntos durante todo el proyecto:** Los **responsables de negocio** (no técnicos / cliente) y los **desarrolladores** (técnicos) deben trabajar juntos diariamente durante todo el proyecto.
5. **Desarrollamos proyectos en torno a individuos motivados:** Los proyectos se construyen en torno a individuos motivados. Hay que darles el entorno, el apoyo necesario y confiar en la ejecución de su trabajo.
6. **El método más eficiente de comunicar información es con conversaciones cara a cara:** El método más eficiente y efectivo de comunicar información es la conversación cara a cara.
7. **La mejor métrica de progreso es el software funcionando:** Es la medida principal y la mejor métrica de progreso, para saber si el mismo está bien encaminado.
8. **Los procesos ágiles promueven el desarrollo sostenible:** Los procesos ágiles promueven un ritmo de trabajo sostenible e indefinido a lo largo del tiempo, evitando el agotamiento del equipo.
9. **La atención continua a la excelencia técnica y al buen diseño mejora la agilidad:** La calidad del producto no es un aspecto negociable. Se deben atender primordialmente a las necesidades mas importantes del cliente.
10. **La simplicidad es esencial:** No se deben agregar funcionalidades extra que el cliente no pidió.
11. **Las mejores arquitecturas, diseños y requerimientos surgen de equipos auto-organizados:** Las mejores arquitecturas, requerimientos y diseños surgen de equipos que tienen libertad para tomar decisiones de forma colaborativa. Los integrantes del equipo deben ser proactivos, aportando soluciones, conceptos y herramientas.
12. **A intervalos regulares, el equipo evalúa su desempeño y ajusta la manera de trabajar:** A intervalos regulares, el equipo reflexiona sobre su

desempeño (retroalimentación) y busca formas de ser más efectivo, ajustando su comportamiento y sus procesos en consecuencia.

Gestión de Producto

Un producto de software se construye para satisfacer una necesidad específica basada en requerimientos, pero el enfoque principal debe estar en el **valor agregado para el cliente**. Se sabe que históricamente alrededor del 80% de las características desarrolladas rara vez o nunca se usan, por lo que el desafío es encontrar ese porcentaje crítico que sí aporta valor. La evolución de un producto se mide en una pirámide: comienza siendo **Funcional** (útil) y **Confiable**, luego pasa a ser **Usable** (productivo para el usuario), y las empresas buscan cruzar la barrera para que sea **Conveniente, Placentero y Significativo**. Para validar el aprendizaje con los clientes invirtiendo el menor esfuerzo posible, se utiliza el **MVP (Producto Mínimo Viable)**, el cual permite probar hipótesis de valor y crecimiento directamente con el comportamiento real de los usuarios.